

شرح ملف

PendulumMotion.cs

البندول ثلاثي الأبعاد — شرح كامل سطرًا بسطر

فيزياء | كود | معادلات | أسئلة محتملة

1. نظرة عامة على الملف

هو المسؤول الوحيد عن تحريك الدلو في المحاكاة. جميع الملفات الأخرى تقرأ منه البيانات ولا تحرك الدلو بنفسها.

ملف PendulumMotion.cs

ماذا يفعل الملف؟

- يحسب موضع الدلو في الفضاء ثلاثي الأبعاد 50 مرة في كل ثانية
- يطبق معادلات الفيزياء: جاذبية + مقاومة هواء + ريح + طفو
- يتتبع الوزن المتناقص مع تسرب الطلاء من الدلو
- يكتشف ارتخاء الحبل أو انقطاعه ويحول الدلو للسقوط الحر
- يصدر السرعة والزاوية لنظام الطلاء في كل frame
- يقيس الدورة الزمنية ويقارنها بالقيمة النظرية للتحقق

جدول العلاقات مع الملفات الأخرى

الملف الآخر	ما يأخذه من PendulumMotion	يستخدمه لـ
PaintPhysics.cs	velocity, angleX/Z, L	حساب سرعة واتجاه جزيئات الطلاء
SimulationManager.cs	currentTension, totalEnergy, displayMass	عرض البيانات وتصديرها CSV/JSON

2. المتغيرات والمعاملات — شرح كل سطر

أ. قسم الحبل

المتغير	القيمة الافتراضية	المعنى والتأثير
L	5 متر	طول الحبل. يزيد → الحركة أبطأ والدورة أطول $(T=2\pi\sqrt{L/g})$
ropelsElastic	false	إذا فعل يصبح الحبل مرنا ويستخدم ropeStiffness
ropeStiffness	5000	صلابة الحبل N/m. يعمل فقط إذا ropelsElastic=true
ropeBreakTension	0	قوة الشد التي ينقطع عندها الحبل. القيمة 0 = لا ينقطع أبدا
ropeBroken	false	قراءة فقط. يصبح true تلقائيا لحظة انقطاع الحبل

ب. قسم الدلو والطلاء

المتغير	النطاق	المعنى
emptyMass	0.3 – 2 kg	وزن الدلو فارغا تماما
initialPaintMass	0.5 – 10 kg	كمية الطلاء في بداية المحاكاة
flowRate	0.001 – 0.1 kg/s	معدل تسرب الطلاء. 0.05 = 50 غرام كل ثانية

ج. قسم البيئة

المتغير	النطاق	المعنى
g	1.6 – 24 m/s ²	الجاذبية. 1.6=قمر / 9.81=أرض / 24=كوكب ضخمة
airDensity	0.5 – 1.5	كثافة الهواء. تزيد → مقاومة أعلى
dragCoef	0.8 – 1.2	معامل جر الهواء. صفر = لا مقاومة. 1 = طبيعي
area	0.02 – 0.1 m ²	مساحة سطح الدلو المعرضة للهواء
windVel	Vector3	سرعة الريح. x=أمام/خلف z=يمين/يسار
useBuoyancy	true/false	قوة الطفو — الهواء يخفف الجاذبية الفعالة
damping	0 – 0.5	تخميد إضافي يشبه الاحتكاك الداخلي في الحبل

د. المتغيرات الخاصة (private) — لا تظهر في Unity

المتغير	المعنى
thetaX, thetaZ	الزاوية الحالية بالراديان على كل محور
angVelX, angVelZ	السرعة الزاوية الحالية rad/s على كل محور
elapsed	الوقت الكلي منذ آخر Reset — لحساب الوزن المتناقص
yOffset	موضع y للدلو نسبة للمحور — يستخدم في حساب الشد
slackPos, slackVel	موضع وسرعة الدلو أثناء السقوط الحر
lastAccX, lastAccZ	التسارع الزاوي آخر خطوة لدالة GetTangentialAcceleration

3. دالة mass — الوزن المتغير مع الزمن

محسوبة — في كل مرة تنادى بها تحسب القيمة من جديد. هذا يعني أن الوزن يتغير تلقائيا مع مرور الوقت دون الحاجة لتحديث يدوي. هذا ليس متغيرا عاديا بل Property

```
public float mass
{
    get
    {
        float m = emptyMass + initialPaintMass - flowRate * elapsed;
        return Mathf.Max(m, Mathf.Max(emptyMass, 0.01f));
    }
}
```

شرح كل سطر:

السطر الأول:

```
float m = emptyMass + initialPaintMass - flowRate * elapsed;
```

الوزن الكلي = وزن الدلو الفارغ + الطلاء الابتدائي - (معدل التسرب × الوقت المنقضي)

مثال رقمي بفرض $emptyMass=1$ و $initialPaintMass=5$ و $flowRate=0.05$:

الوقت	الحساب	الوزن
ثانية 0 = t	$1.0 + 5.0 - 0.05 \times 0$	6.0 kg
ثانية 20 = t	$1.0 + 5.0 - 0.05 \times 20$	5.0 kg
ثانية 60 = t	$1.0 + 5.0 - 0.05 \times 60$	3.0 kg
ثانية 100 = t	$1.0 + 5.0 - 0.05 \times 100$	1.0 kg (emptyMass يتوقف عند)

السطر الثاني:

```
return Mathf.Max(m, Mathf.Max(emptyMass, 0.01f));
```

يمنع الوزن من الوصول لصفر أو أقل. الحد الأدنى هو emptyMass دائما.

الجواب: Mathf.Max تضمن أن الوزن لا ينزل أقل من emptyMass إذا سألك الأستاذ: لماذا الوزن لا يصل صفرا؟ *

4. دالة ResetSimulation — إعادة التشغيل

تستدعى في حالتين: عند بدء اللعبة (Start)، وعند ضغط زر Reset في الواجهة.

```
public void ResetSimulation()
{
    thetaX = initialAngleDeg * Mathf.Deg2Rad;

    thetaZ = 0f;

    thetaX0 = thetaX; thetaZ0 = thetaZ;

    angVelX = 0f;

    angVelZ = initialAngVel;

    elapsed = 0f;

    ropeIsSlack = false; ropeBroken = false;

    UpdatePositionFromAngles();

    prevPos = transform.position;
}
```

شرح كل سطر:

السطر	المعنى
<code>thetaX = initialAngleDeg * Mathf.Deg2Rad</code>	تحويل الزاوية من درجات إلى راديان لأن المعادلات تستخدم الراديان
<code>thetaX0 = thetaX</code>	حفظ الزاوية الابتدائية لاستخدامها لاحقاً في حسابات التحقق
<code>angVelX = 0f / angVelZ = initialAngVel</code>	الحركة تبدأ بسرعة صفر على X وبالسعة المحددة على Z
<code>elapsed = 0f</code>	تصغير عداد الوقت حتى تبدأ معادلة الوزن من جديد
<code>ropeBroken = false / ropeIsSlack = false</code>	إعادة حالة الحبل لوضعه الطبيعي
<code>UpdatePositionFromAngles()</code>	ضبط الموضع الجسدي للدلو في Unity بناءً على الزاوية الجديدة

5. دالة FixedUpdate — القلب النابض

ثابت 50 مرة في الثانية (كل 0.02 ثانية). استخدام FixedUpdate بدل Update ضروري لأن المعادلات الفيزيائية تحتاج dt ثابتاً. تستدعى بشكل

```
void FixedUpdate()
{
    float dt = Time.fixedDeltaTime; // = 0.02 ثواني
    if (dt <= 0f) return; // رفض يلع دم سقلا نم ةيامح
    elapsed += dt; // تقولا دادع شي دحت

    if (impulseQueued) // اذ! طغض
    {
        angVelX += impulse;
        angVelZ += impulse;
        impulseQueued = false;
    }

    // لبحلا ةلاح يلع ءانب ةكرحلا عون راي تخا
    Vector3 newPos = (ropeBroken || ropeIsSlack)
        ? StepFreeFall(dt) // وخر وأ عطقنم لبح
        : StepPendulum(dt); // ميس لبح

    velocity = (newPos - prevPos) / dt; //  $v = \Delta x / \Delta t$ 
    transform.position = newPos;
    prevPos = newPos;
    UpdateReadouts();
    if (validationMode) ValidateStep(dt);
}
```

أهم نقطة: حساب السرعة

السرعة تحسب من الفرق بين الموضعين القديم والجديد:

$$\text{velocity} = (\text{newPos} - \text{prevPos}) / dt$$

هذه السرعة يقرأها PaintPhysics لتحديد السرعة الابتدائية لجزيئات الطلاء.

(fps) → نتائج فيزيائية مستقرة وقابلة للتكرار. Update يعمل بمعدل متغير (60 + fps تبعاً للأداء). لماذا FixedUpdate وليس Update؟ *
FixedUpdate يعمل بمعدل ثابت (50)

6. دالة StepPendulum — المحرك الفيزيائي الرئيسي

هذه أهم دالة في الملف. تحسب الموضع الجديد للدلو بناء على قوى فيزيائية حقيقية. نشرحها خطوة بخطوة:

الخطوة 1 — الجاذبية الفعالة

```
float gEff = useBuoyancy  
  
? g * (1f - (airDensity * bucketVolume) / m)  
  
: g;
```

يطبق قانون أرخميدس: الهواء يخفف الجاذبية بمقدار يتناسب مع حجم الدلو وكثافة الهواء. بخلاف ذلك تستخدم g المحددة مباشرة. إذا كان `useBuoyancy=true`

الخطوة 2 — معامل مقاومة الهواء

```
float kd = (airDensity * dragCoef * area * L) / (2f * m);
```

يتغير مع الزمن. كلما خف الدلو كلما زاد تأثير الهواء عليه m لأن `frame` هو معامل التخميد الهوائي. يعاد حسابه في كل `kd`

الخطوة 3 — التسارع الزاوي (أهم معادلة في الملف)

```
float accX = -(gEff / L) * Mathf.Sin(thetaX)  
  
- kd * angVelX * Mathf.Abs(angVelX)  
  
- damping * angVelX;
```

هذه معادلة البندول الكاملة. كل جزء له معنى فيزيائي:

التأثير	المعنى الفيزيائي	الجزء في الكود
يسبب التآرجح الأساسي	قوة الجاذبية تسحب الدلو للمركز	$-(gEff/L) \cdot \sin(\theta)$
تبطئ الحركة تدريجيا	مقاومة هواء تربيعية في السرعة	$kd \cdot \omega \cdot \omega $
تبطئ الحركة بشكل خطي	تخميد خطي (احتكاك داخلي)	$damping \cdot \omega$

قيمة موجبة، لكن $|\omega| \cdot \omega$ يحافظ على إشارة ω الأصلية. لأن المقاومة الهوائية تعاكس دائما اتجاه الحركة. لماذا $|\omega| \cdot \omega$ وليس $kd \cdot \omega^2$ فقط؟ ω^2 يعطي دائما

الخطوة 4 — التكامل العددي (Semi-implicit Euler)

```
angVelX += accX * dt; // الوداءة إرسال شي دحت //
```

```
thetaX += angVelX * dt; // ةدي دجل اة عرسل اب عضوم ل ا م ث
```

الطريقة	الترتيب	الاستقرار
Explicit Euler (البسيطة)	موضع بالسرعة القديمة ← ثم السرعة	غير مستقرة مع dt كبير
Semi-implicit Euler (المستخدمة)	سرعة أولا ← موضع بالسرعة الجديدة	أكثر استقرارا بنفس التكلفة

الخطوة 5 — تحويل الزوايا إلى موضع 3D

```
float x = L * Mathf.Sin(thetaX);  
float z = L * Mathf.Sin(thetaZ);  
float y = -Mathf.Sqrt(Mathf.Max(0f, L*L - x*x - z*z));
```

هندسة كروية: الدلو يقع دائما على سطح كرة نصف قطرها L مركزها نقطة التعليق:

$$x = L \cdot \sin(\theta X) \quad z = L \cdot \sin(\theta Z) \quad y = -\sqrt{(L^2 - x^2 - z^2)}$$

الـ y سالب دائما لأن الدلو يتدلى تحت نقطة التعليق.

الخطوة 6 — فحص حالة الحبل

```
currentTension = GetTensionForce();  
  
if (ropeBreakTension > 0f && currentTension > ropeBreakTension)  
{  
    ropeBroken = true;  
    slackVel = velocity;  
    slackPos = new Vector3(x, y, z);  
}  
  
else if (currentTension <= 0f)  
{  
    ropeIsSlack = true;  
    slackVel = velocity;  
    slackPos = new Vector3(x, y, z);  
}
```

عند الانقطاع أو الارتخاء يحفظ الموضع والسرعة الحالية كنقطة بداية للسقوط الحر.

الخطوة 7 — مرونة الحبل

```
float r = L;

if (ropeIsElastic && ropeStiffness > 0f)

    r = L + Mathf.Max(0f, currentTension) / ropeStiffness;

return pivot.position + (new Vector3(x,y,z) / L) * r;
```

إذا كان الحبل مرنا يضاف تمدد بناء على قانون هوك:

$$r = L + T / \text{ropeStiffness}$$

7. دالة StepFreeFall — السقوط الحر

تفعل عندما يكون الحبل منقطعاً أو رخوا. الدلو يتحرك تحت الجاذبية فقط.

```
Vector3 StepFreeFall(float dt)

{
    slackVel += Vector3.down * g * dt; // لفسألل ةرسلال ديزت ةيبدأجلال
    slackPos += slackVel * dt; // عضومال شي دحت

    if (!ropeBroken && slackPos.magnitude >= L) // ؟ اددجم لبجلال ءدش له
    {
        Vector3 dir = slackPos.normalized;

        slackPos = dir * L;

        thetaX = Mathf.Asin(Mathf.Clamp(slackPos.x / L, -1f, 1f));
        thetaZ = Mathf.Asin(Mathf.Clamp(slackPos.z / L, -1f, 1f));

        angVelX = slackVel.x / (L * Mathf.Cos(thetaX));
        angVelZ = slackVel.z / (L * Mathf.Cos(thetaZ));

        ropeIsSlack = false; // لودن بلال عضول ةدوعلال
    }

    return pivot.position + slackPos;
}
```

لماذا نتحقق من $\text{slackPos.magnitude} \geq L$ ؟

إلى L مجدداً (يعني الحبل امتد بالكامل) يعني الحبل شد فجأة. عندها نحول السرعة الخطية إلى سرعة زاوية ونعود لحسابات البندول. عندما يرتخي الحبل يسقط الدلو حراً. إذا وصلت مسافته من نقطة التعليق

هذا الانتقال يسبب نبضة قوية في الحركة — وهو سلوك فيزيائي صحيح تماماً لأن الحبل يشد فجأة مما يعطي الدلو دفعة. □

8. دالة GetTensionForce — حساب قوة الشد

في الحبل في أي لحظة. تستدعى في كل frame من StepPendulum وأيضا من SimulationManager للعرض والتصدير. تحسب قوة الشد

```
public float GetTensionForce()
{
    float cosEff = -yOffset / L;

    float centripetal = L * (angVelX*angVelX + angVelZ*angVelZ);

    return mass * (g * cosEff + centripetal);
}
```

المعادلة:

$$T = m \cdot (g \cdot \cos(\theta_{\text{eff}}) + L \cdot \omega^2)$$

الجزء	المعنى
$\cos\text{Eff} = -y\text{Offset}/L$	جيب تمام الزاوية الفعالة. مشتق هندسيا من موضع y للدلو
$g \cdot \cos\text{Eff}$	الجزء المسبب بالجاذبية — أعلى قيمة عند النقطة الأدنى ($\theta=0$)
$L \cdot (\omega X^2 + \omega Z^2)$	القوة المركزية (centripetal) — تزيد مع سرعة التأرجح
$\text{mass} \cdot (...)$	ضرب الكتلة لإيجاد قوة الشد بالنيوتن

لأن $\cos\text{Eff}=1$ (أعلى قيمة) وسرعة التآرجح في أقصاها أيضا. عند النقطة الأدنى من مسار الدلو ($\theta=0$). متى يكون الشد في أعلى قيمته؟ *

9. دالة UpdateReadouts — تحديث القراءات

تستدعى في نهاية كل FixedUpdate لحساب المتغيرات المعروضة في الواجهة.

```
void UpdateReadouts()
{
    float m = mass;

    float vx = L * angVelX; //  $v_x = L \times \omega_x$ 
    float vz = L * angVelZ;

    kineticEnergy = 0.5f*m*vx*vx + 0.5f*m*vz*vz; //  $KE = \frac{1}{2}mv^2$ 
    potentialEnergy = m * g * yOffset; //  $PE = mgy$ 
    totalEnergy = kineticEnergy + potentialEnergy;

    theoreticalPeriod = 2f * Mathf.PI * Mathf.Sqrt(L/g); //  $T = 2\pi\sqrt{L/g}$ 

    angleX = thetaX * Mathf.Rad2Deg; //  $\theta_x \rightarrow$  زاوية الدوران
    angleZ = thetaZ * Mathf.Rad2Deg;

    displayMass = m;
}
```

الكمية	المعادلة	الوحدة
الطاقة الحركية	$KE = \frac{1}{2}m(v_x^2 + v_z^2)$	جول J
طاقة الوضع	$PE = m \cdot g \cdot y$ (سالبة تحت المحور y)	جول J
الطاقة الكلية	$E = KE + PE$	جول J
دورة التآرجح	$T = 2\pi\sqrt{L/g}$	ثانية s

= خطأ في المعاملات. إذا تناقصت بسرعة dragCoef مرتفع جداً. الطاقة الكلية يجب أن تتناقص ببطء مع الزمن بسبب مقاومة الهواء. □
إذا تزايدت

10. دالة ValidateStep — التحقق من الصحة

تعمل فقط إذا كان `validationMode=true`. تقيس الدورة الفعلية وتقارنها بالقيمة النظرية وتطبع التقرير كل 3 ثوان في Console.

```
void ValidateStep(float dt)
{
    if (totalEnergy < vEnergyMin) vEnergyMin = totalEnergy;
    if (totalEnergy > vEnergyMax) vEnergyMax = totalEnergy;
    if (currentTension > maxTensionObserved) maxTensionObserved = currentTension;

    // بجوملل بلاسلدا نم رفعلدا θX زواجتي ةرم لك ليحست :ةرودلدا سايق
    float s = Mathf.Sign(thetaX);
    if (s > 0f && lastThetaXSign <= 0f)
    {
        if (lastCrossTime > 0f) measuredPeriod = Time.time - lastCrossTime;
        lastCrossTime = Time.time;
    }
    lastThetaXSign = s;

    vLogTimer += dt;
    if (vLogTimer >= 3f) // 3 ثاوت لك
    {
        vLogTimer = 0f;
        Debug.Log($"T_measured={measuredPeriod:F3}s "
            + $"vs T_theory={theoreticalPeriod:F3}s | "
            + $"E_drift={vEnergyMax-vEnergyMin:F3}J | "
            + $"Tension_max={maxTensionObserved:F2}N");
    }
}
```

ما يقاس	المعنى	القيمة المثالية
T_measured vs T_theory	الدورة المقاسة مقابل $(2\pi\sqrt{L/g})$	فرق أقل من 5%

E_drift	تذبذب الطاقة الكلية (E_max - E_min)	صغير قدر الإمكان
Tension_max	أقصى شد ملاحظ مقابل المعادلة التحليلية	متقاربان

11. كيف يتصل الملف مع بقية المشروع

PaintPhysics.cs تقرأ من PendulumMotion:

المتغير / الدالة	كيف تستخدمه PaintPhysics
bucketMotion.velocity	السرعة الابتدائية لجزيئات الطلاء عند الانبعاث
bucketMotion.angleX / angleZ	لحساب زاوية ميل الدلو وتأثيره على التدفق
bucketMotion.L	في معادلة $\omega = v/L$
bucketMotion.GetTangentialAcceleration()	لحساب اضطراب الطلاء داخل الدلو (slosh)

SimulationManager.cs يقرأ من PendulumMotion:

المتغير	يستخدم في
pendulum.angleX	رسم تاريخ الزاوية وتصديره
pendulum.currentTension	عرض الشد في Output + تصديره
pendulum.totalEnergy	رسم تاريخ الطاقة
pendulum.displayMass	عرض Mass m(t) في الواجهة
pendulum.theoreticalPeriod	عرض Period في الواجهة
pendulum.L + باقي المعاملات	تصدير (SimReport JSON)

12. أسئلة محتملة من الأستاذ وإجاباتها

أين طبقتم معادلة البندول في الكود؟

في دالة StepPendulum، السطر الأساسي:

```
accX = -(gEff/L)*Mathf.Sin(thetaX) - kd*angVelX*Mathf.Abs(angVelX) -  
damping*angVelX;
```

هذه هي المعادلة: $\theta'' = -(g/L) \cdot \sin(\theta) - \text{drag} - \text{damping}$

كيف يتغير وزن الدلو مع الزمن؟

عبر Property تسمى mass:

```
float m = emptyMass + initialPaintMass - flowRate * elapsed;
```

تمنع الوزن من الوصول لصفر Mathf.Max FixedUpdate. يزيد في كل elapsed.

ما هي طريقة التكامل العددي المستخدمة؟

Semi-implicit Euler (Symplectic Euler):

```
angVelX += accX * dt; // الأول إرسال  
thetaX += angVelX * dt; // الثاني إرسال
```

أفضل من Explicit Euler من ناحية الاستقرار بنفس التكلفة الحسابية.

كيف تكتشفون انقطاع الحبل؟

في نهاية StepPendulum:

```
if (ropeBreakTension > 0f && currentTension > ropeBreakTension)  
ropeBroken = true;
```

بعدها FixedUpdate يستدعي StepFreeFall بدلا من StepPendulum.

كيف تحسبون قوة الشد في الحبل؟

دالة GetTensionForce():

```
T = mass * (g * cosEff + L * (angVelX2 + angVelZ2))
```

مشتق هندسيا من موضع الدلو ← $\cos\text{Eff} = -y\text{Offset}/L$
الجزء الثاني هو القوة المركزية (centripetal).

لماذا نستخدمون FixedUpdate بدل Update؟

لأن المعادلات الفيزيائية تحتاج dt ثابتا.
يعمل بمعدل متغير تبعا للأداء Update
نتائج مستقرة وقابلة للتكرار $\rightarrow$ fps يعمل بمعدل ثابت FixedUpdate 50

لماذا ropeStiffness لا يؤثر أحيانا؟

لأن الكود يتحقق من ropeIsElastic أولا:

```
if (ropeIsElastic && ropeStiffness > 0f)
```



```
r = L + Mathf.Max(0f, currentTension) / ropeStiffness;
```


يجب تفعيل Elastic rope checkbox أولا حتى يعمل ropeStiffness.

كيف نتحقق من صحة نتائج المحاكاة؟

دالة ValidateStep() عند تفعيل validationMode:
تقيس الدورة الفعلية وتقارنها بـ $2\pi\sqrt{L/g}$
تتبع تذبذب الطاقة الكلية ($E_{\text{max}} - E_{\text{min}}$)
تقارن أقصى شد ملاحظ بالقيمة التحليلية
تطبع التقرير كل 3 ثوان في Console.

كيف يعرف نظام الطلاء سرعة الدلو؟

في FixedUpdate:

```
velocity = (newPos - prevPos) / dt;
```


هذا المتغير public يقرأه PaintPhysics.cs مباشرة:

```
Vector3 vel = bucketMotion.velocity + exitDirection*speed + spread;
```

ما الفرق بين ropeBroken و ropeIsSlack؟

ropeBroken = true ← الحبل انقطع نهائيا ولا يعود أبدا

ropeIsSlack = true ← الحبل ارتخى مؤقتا ويعود عندما يشد مجددا

كلاهما يحول الحساب إلى StepFreeFall، لكن StepFreeFall

يتحقق من ropeIsSlack فقط لإعادة الحبل للعمل.

ورقة المراجعة السريعة

ضعها أمامك أثناء النقاش مع الأستاذ

السؤال المتوقع	الدالة / السطر في الكود
معادلة البندول؟	StepPendulum $\leftarrow \text{accX} = -(g_{\text{Eff}}/L) * \sin(\theta) - \text{drag} - \text{damping}$
الوزن المتغير؟	Property mass $\leftarrow \text{emptyMass} + \text{initialPaintMass} - \text{flowRate} * \text{elapsed}$
طريقة التكامل؟	Semi-implicit Euler $\leftarrow$ سرعة أولا ثم موضع
حساب الشد؟	GetTensionForce $\leftarrow T = m * (g * \cos \text{Eff} + L * \omega^2)$
انقطاع الحبل؟	StepPendulum $\leftarrow$ if (currentTension > ropeBreakTension)
السقوط الحر؟	StepFreeFall $\leftarrow \text{slackVel} += \text{Vector3.down} * g * dt$
التحقق من النتائج؟	ValidateStep $\leftarrow T_{\text{measured}}$ vs $2\pi\sqrt{L/g}$ كل 3 ثوان
سرعة الدلو لنظام الطلاء؟	FixedUpdate $\leftarrow \text{velocity} = (\text{newPos} - \text{prevPos}) / dt$
لماذا FixedUpdate؟	ثابت = استقرار فيزيائي dt
ropeStiffness لا يعمل؟	يحتاج ropeIsElastic=true أولا